

Localizing and Beating the Out-of-Distribution Bottleneck in Vector Search: A Measured Collection of Routing, Coverage, and Systems Techniques

[TODO: authors]

Abstract

Scalable approximate nearest-neighbor (ANN) search routes a query to a few partition cells, scans quantized codes inside them, and re-ranks a shortlist. On *out-of-distribution* (OOD) workloads — where queries come from a different distribution than the database, as in cross-modal (text→image) retrieval under inner product — we first *localize the bottleneck*: a battery of scan-side and code-side improvements (wider SIMD, higher-resolution and rotated codes, RaBitQ, ADC routing) leave OOD recall/QPS essentially unmoved, while the achievable recall is capped by which cells get *routed to and covered*. We then present a collection of routing- and coverage-directed techniques, each measured in isolation: a one-parameter routing recalibration (γ) grounded in a cap-versus-ball selectivity argument; k -NN-graph coverage augmentation; a joint cross-level partition optimization; multi-assignment; and a corollary that *reverses* standard practice — once graph coverage is present, coarser cells win. Composed and evaluated under a same-hardware, tight-pairwise protocol against ScaNN at its official configuration, the engine attains higher QPS at matched recall@10 at 1M, 10M[[pending: , and 100M](#)], and transfers to the streaming track. We report the negative results as first-class evidence for the localization, and treat the measurement methodology — which repeatedly caught mirages during development — as a contribution.

1 Introduction

Approximate nearest-neighbor (ANN) search is the retrieval primitive under retrieval-augmented generation, semantic search, and recommendation. At scale the dominant family is inverted-file with quantization (IVF-PQ): partition the database into C cells, at query time *route* to the $p \ll C$ most promising cells, *scan* compressed codes within them, and exactly *re-rank* a shortlist. A large literature has driven each stage — anisotropic quantization and SIMD fast-scan for the scan, product and residual codes for memory, graph indices as an alternative to IVF — overwhelmingly evaluated where queries and database are drawn from the *same* distribution.

We study the *out-of-distribution* (OOD) regime, where the query distribution differs from the database. This is not a corner case: cross-modal retrieval embeds a text query and searches image vectors under inner product, and the big-ANN “text2image” track (image base, text queries, $d=200$) is exactly this setting. It is also where the strongest systems separate, and — we argue — where the conventional intuition about what to optimize is wrong.

Our organizing claim is a *bottleneck localization*: on OOD workloads the binding constraint is *routing coverage* — whether the true neighbor’s cell is among the p probed and its members reachable — not scan throughput or code fidelity. We support this from both sides. A battery of scan- and code-side improvements (wider SIMD, higher-resolution and rotated codes, RaBitQ, ADC routing) leaves OOD recall and QPS essentially unmoved (§12); meanwhile a small set of routing- and coverage-directed techniques each yields a measured gain (§6–§10). We therefore present the work as a measured collection — every technique reported with the effect it produces in isolation, and the negatives kept as first-class evidence for the localization. Composed and evaluated under a same-hardware, tight-pairwise protocol against ScaNN at its best configuration, the result attains higher QPS at matched recall@10 at 1M and 10M [[pending: \(and 100M\)](#)], and transfers to the streaming track.

Contributions (each measured in isolation).

- **A bottleneck localization** (§5): scan- and code-side levers do not move OOD; routing/coverage does. The negatives (§12) are the evidence.
- **γ routing recalibration** (§6), grounded in a cap-vs-ball selectivity argument (§4). *Effect: $\approx$ half the probes at matched OOD recall, zero query-time cost.*
- **k -NN-graph coverage** (§7). *Effect: composes with γ ; at 1M moves the ScaNN ratio $1.18 \rightarrow 0.98$.*
- **The coarse-cell corollary** (§8). *Effect: with graph coverage present, coarser cells win; at 10M $1.01 \rightarrow 0.85$.*
- **Joint cross-level partition optimization (tree-EM)** (§9). *Effect: $+0.6$ – 1.0 recall@10 points at 10M, QPS-neutral.*
- **Multi-assignment (SOAR)** (§10). *Effect: raises the routing recall ceiling to baseline parity.*
- **Systems levers for streaming** (§11): batch-parallel insert ($4.5\times$), low-live adaptive p (worst step $0.74 \rightarrow 0.94$), and a parallelization fix ($7.6\times$ at 8 threads).
- **A same-hardware evaluation methodology** (§13) and an honest scope of what the ratios do and do not claim.

2 Background: Standard Building Blocks

Our system is assembled from well-established components; we summarize them here so the contributions in §3–§10 can be stated as deltas against known practice.

Inverted file (IVF). Partition the database into C cells (the Voronoi regions of C centroids). At query time, *route* the query to its $p \ll C$ nearest centroids and scan only those cells. End-to-end recall is capped by whether the true neighbor’s cell is among the p probed — the routing question that this paper argues is the OOD bottleneck.

Hierarchical routing. Scoring all C centroids per query is $O(Cd)$, which dominates at scale (fine C is needed to keep cells small). A *tree* of centroids — coarse k -means over C_0 centroids, then fine k -means within each — reduces routing to $O(LC^{1/L})$ for an L -level tree (FAISS uses an HNSW graph over the centroids for the same purpose). We use hierarchical k -means.

Product quantization (PQ). Split each vector’s d dimensions into m subspaces and quantize each subspace to a small codebook (16 codewords for 4-bit PQ). Approximate distances are computed by lookup-and-add over per-subspace tables (ADC). With 4-bit codes the table lookup is a register shuffle (`vpshufb`), enabling the SIMD “fast-scan” kernels (Quick-ADC, ScaNN, FAISS FastScan). *Anisotropic* / score-aware quantization (ScaNN) weights the quantization loss to preserve inner-product ranking rather than reconstruction.

Exact re-ranking. The quantized scan is used only to select a shortlist; exact distances (int8 or float, read for the few shortlisted vectors) re-order it. The shortlist depth t trades recall for cost.

Multi-assignment. Replicating each point into its top- a_0 cells (a space/time tradeoff) raises the chance a query’s neighbor lands in a probed cell.

The OOD workload. The big-ANN OOD track (text2image) pairs *image*-embedding base vectors ($d=200$) with *text*-embedding queries under inner product, with a float ground truth. Base norms are concentrated but not constant (≈ 0.81 – 0.99); queries are unit-norm. This distribution gap between base and queries is what breaks L_2 -trained routing (§6).

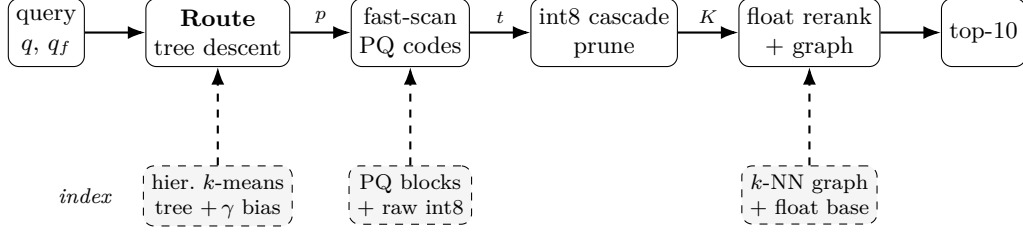


Figure 1: Query pipeline (top) and the offline-built index it consumes (bottom, dashed). Edge labels are the shrinking candidate set: p probed leaves $\rightarrow$ top- t pool $\rightarrow K$ cascade survivors $\rightarrow$ top-10. γ calibration, the k -NN graph union, and the coarse-cell tree geometry are this work’s contributions (Table 1).

3 System Overview

Figure 1 shows the pipeline; we describe the build and query paths, then delineate standard vs. contributed components (Table 1).

Build path.

1. **Quantize.** Float base $\rightarrow$ int8 via a single global scale ($127/\max|x|$); the int8 base is the native scan/route representation.
2. **Train the routing tree.** Hierarchical k -means: C_0 coarse centroids, then fine centroids within each coarse cell to K_f leaves total (2-level; generalizes to L levels).
3. **Joint refinement (tree-EM, §9).** Optionally re-optimize all levels jointly (E: reassign sample points to their nearest leaf via the beam descent; M: recompute every level’s centroids).
4. **Multi-assign (SOAR, §10).** Assign each point to a_0 leaves with spill that covers the residual direction of the first.
5. **Encode.** 4-bit anisotropic PQ ($m=d/2$ subspaces); pack codes into cell-contiguous 16-row blocks laid out for the `vpshufb` fast-scan. Store the raw int8 vectors in cell order too (cache-warm re-rank gathers).
6. **Calibrate & augment.** Precompute the per-cell γ routing bias (§6); build the base-side k -NN graph sidecar (§7).

Query path.

1. **Route (tree descent).** Score the C_0 coarse centroids (int8, VNNI kernel), keep the top- b_0 ; expand to their fine children, score those with the γ bias added, and return the top- p leaves.
2. **Scan.** Fast-scan the 4-bit PQ codes of the p probed cells (`vpshufb` ADC with int16 accumulation for ~ 12 -bit selection resolution; optional AVX-512 64-wide), collecting a bounded top- t pool.
3. **Cascade re-rank.** Deduplicate the pool by original id (needed under multi-assignment); exact-rescore the t survivors in int8 (VNNI dot); prune to the top- K ($K=16$).
4. **Float re-rank + graph union.** Compute exact float inner products for the K survivors (reading the original float vectors) unioned with the k -NN graph neighbors of the top- M ; return the top-10.

Algorithms 1–3 state the pipeline precisely. All routing scores use the calibrated form $s_\gamma(q, c) = \gamma\|c\|^2 - 2q \cdot c$ (§6); the constant $\|q\|^2$ is dropped, and smaller s_γ means a better cell. Leaf centroids are $\{c_j\}_{j=1}^{K_f}$ with precomputed bias $b_j = (\gamma - 1)\|c_j\|^2$ so that $s_\gamma(q, c_j) = \|c_j\|^2 - 2q \cdot c_j + b_j$ is one dot product plus an additive constant.

Algorithm 1: Build

Input: float base $X_f \in \mathbb{R}^{n \times d}$; tree sizes C_0, K_f ; multi-assign a_0 ; calibration γ ; EM rounds R ; graph degree k

Output: index $(\{Q_0\}, \{c_j\}, b, \text{PQ blocks, raw}, G)$

```
1  $\sigma \leftarrow 127 / \max_i \|X_f[i]\|_\infty$ ;  $X \leftarrow \text{int8}(\sigma X_f)$ ; // global-scale quantize
2  $\{Q_0\} \leftarrow k\text{-means}(X, C_0)$ ;  $\{c_j\} \leftarrow \text{fine } k\text{-means within each coarse cell to } K_f \text{ leaves}$ ;
3 for  $R$  rounds // tree-EM (§9); optional do
4   E: assign each sample to its nearest leaf via  $\text{Route}(\cdot, 1)$ ;
5   M: recompute  $\{Q_0\}, \{c_j\}$  from the assignments;
6 foreach point  $x_i$  // SOAR multi-assign (§10) do
7   assign  $x_i$  to its top- $a_0$  leaves; spill covers the residual of the 1st;
8 per leaf: encode members as 4-bit anisotropic PQ, pack into 16-row vpshufb blocks; store raw int8
   in leaf order;
9  $b_j \leftarrow (\gamma - 1) \|c_j\|^2$  for all  $j$ ; // routing bias
10  $G \leftarrow \text{base-side } k\text{-NN graph (self-search; §7)}$ ;
11 return index  $(\{Q_0\}, \{c_j\}, b, \text{PQ blocks, raw}, G)$ ;
```

Algorithm 2: Route (tree descent) – returns the p best leaves

```
1 Procedure  $\text{Route}(q, p)$ :
2    $S_0 \leftarrow \text{top-}b_0 \text{ coarse cells minimizing } s_\gamma(q, Q_0)$ ; // int8/VNNI kernel
3    $\mathcal{L} \leftarrow \text{fine children of } S_0$ ;
4   score each  $c_j \in \mathcal{L}$  by  $\|c_j\|^2 - 2q \cdot c_j + b_j$ ;
5   return the  $p$  leaves of  $\mathcal{L}$  with smallest score;
```

4 Theoretical Grounding (Scoped)

[TODO: (i) Cap-vs-ball under concentration: additive ball filters are non-selective in high dimension (everything within $R + r$), angular/spherical-cap filters remain selective; k -means on centered data realizes a data-dependent angular partition. (ii) This frames γ : the routing score $\gamma \|c\|^2 - 2q \cdot c$ interpolates $\gamma=1$ (additive ball / L_2) to $\gamma \rightarrow 0$ (pure inner product / cap), so the theory predicts $\gamma < 1$ is needed precisely in the concentrated OOD regime – which we confirm. (iii) Two principles we build on: route-precise/rank-cheap, and multi-assignment as a space/time tradeoff. Explicitly state we do NOT claim the recursive-tree polylog result (refuted at $\leq 10\text{M}$ scale).]

5 Localizing the OOD Bottleneck

We separate two ceilings. The *pool-recall ceiling* is the recall obtainable by exactly re-ranking the *entire* probed pool — it depends only on routing (which cells, hence which points, are reachable), not on the scan’s approximation. The *scan-limited recall* is what the actual (quantized) scan plus shortlist delivers. If the two coincide, the scan is already extracting everything the routed pool contains and there is no scan headroom; recall is then a pure function of routing coverage.

On OOD text2image this is what we observe: at a fixed probe budget the scan-limited recall sits at the pool-recall ceiling, and that ceiling — not the scan — is what moves when we change routing. Two consequences follow, and the rest of the paper is their systematic exploration. First, techniques that improve *coverage* (route calibration, graph augmentation, joint partition optimization, multi-assignment) should raise recall; §6–§10 confirm each does, with a measured effect. Second, techniques that only improve the *scan or the code* (wider SIMD, higher-resolution or rotated codes, alternative quantizers, ADC routing) should not help, because there is no scan headroom to recover; §12 confirms each does not. The negatives are thus not incidental — they are the control that localizes the bottleneck to routing coverage.

Algorithm 3: Query

Input: int8 query q , float query q_f ; probes p ; pool depth t ; prune K ; graph fan-in M

```

1  $\mathcal{C} \leftarrow \text{Route}(q, p)$ ;
2  $\mathcal{P} \leftarrow \emptyset$ ; // bounded top- $t$  pool
3 foreach  $cell \in \mathcal{C}$  do
4   fast-scan its 4-bit PQ codes (vpshufb ADC, int16 accum)  $\rightarrow$  approx. scores;
5   keep the  $t$  best in  $\mathcal{P}$  (threshold-bounded collect);
6  $\mathcal{P} \leftarrow$  dedup  $\mathcal{P}$  by original id ; // needed under  $a_0 > 1$ 
7  $\mathcal{K} \leftarrow$  top- $K$  of  $\mathcal{P}$  by exact int8 IP (VNNI dot) ; // cascade prune
8  $\mathcal{R} \leftarrow \mathcal{K} \cup \bigcup_{r \in \text{top-}M(\mathcal{K})} G[r]$  ; //  $k$ -NN-graph coverage union
9 return top-10 of  $\mathcal{R}$  by exact float IP  $\langle q_f, x_r \rangle$ ;
```

Table 1: Components of the pipeline: standard building blocks vs. this work.

Component	Basis	Our delta
int8 IVF partition	standard	—
Hierarchical k -means tree	standard (FAISS-style)	fast int8/VNNI descent kernel
4-bit anisotropic PQ	ScaNN / FastScan family	int16-resolution + 32/64-wide fast-scan
Multi-assignment (SOAR)	ScaNN-derived	dedup-before-cap correctness fix
int8 $\rightarrow$ float re-rank cascade	folklore	the specific $K=16$ composition
γ routing calibration	ours	the OOD miscalibration fix
k -NN-graph coverage union	ours	composition with γ + float re-rank
Coarse-cell geometry knee	ours	reverses “finer is better”
Tree-EM joint optimization	ours	cross-level partition objective
Batch-insert / adaptive- p (streaming)	ours	§11

6 γ : One-Parameter Routing Recalibration

An IVF router ranks cells by a score over the query q and centroid c . For L_2 k -means the score is $\|q - c\|^2 = \|q\|^2 + \|c\|^2 - 2q \cdot c$; dropping the query-constant term leaves $\|c\|^2 - 2q \cdot c$. We write the family

$$s_\gamma(q, c) = \gamma \|c\|^2 - 2q \cdot c, \quad (1)$$

where $\gamma=1$ recovers L_2 routing and $\gamma \rightarrow 0$ recovers pure inner-product (angular) routing. Under distribution shift these disagree in a specific way: the $\|c\|^2$ penalty demotes high-norm centroids, but for OOD queries the inner-product-relevant neighbors concentrate precisely near those high-norm cells, so $\gamma=1$ systematically misroutes them. A single global $\gamma < 1$ rebalances the two terms. It is grounded in the cap-vs-ball argument of §4: γ slides routing along the additive-ball ($\gamma=1$, non-selective under concentration) to spherical-cap ($\gamma \rightarrow 0$, selective) axis.

Crucially the fix is *free at query time*: s_γ differs from the L_2 score only by the per-cell constant $(\gamma-1)\|c\|^2$, which we fold into a precomputed integer bias added during cell scoring (no extra query work, no index-format change). We fit γ once per dataset; $\gamma \approx 0.5$ is optimal on text2image at both 1M and 10M, and the recall is flat across $\gamma \in \{0.4, \dots, 0.6\}$ near the optimum.

Effect. Reaches matched OOD recall at $\approx$ half the probes (p : 54 $\rightarrow$ 29 at 1M; $\approx 2.5\times$ probe cut at 10M). Zero query-time cost.

7 k -NN-Graph Coverage Augmentation

Routing to p cells can still miss a true neighbor whose cell went unprobed. We recover such misses with a graph, without probing more cells. Offline we build a base-side inner-product k -NN graph (a per-point adjacency list; constructed by ScaNN self-search or by the engine querying itself — the two agree on $\sim 92\%$ of edges). At query time, after the scan yields a shortlist, we take the top- M shortlist members and *union*

Table 2: Tree geometry and per-query routing cost (Eq. 2) by scale. Beams are aggressive ($b_0 \approx 12\text{--}25\%$ of coarse cells) because OOD queries route poorly. [\[pending: 100M validated via a scaling-law sweep \(§8.1\).\]](#)

scale	L	K_f	cell counts	beams	route evals	pts/leaf
1M	2	16,384	$C_0=768$	$b_0=96$	$768 + 96 \cdot 21 = 2,816$	61
10M	2	65,536	$C_0=4096$	$b_0=128$	$4096 + 128 \cdot 16 = 6,144$	152
100M	3	524,288	$C_0=2048, C_1=32768$	$b_0=512, b_1=256$	$2048 + 512 \cdot 16 + 256 \cdot 16 = 14,336$	190

their graph neighbors into the exact re-rank set. The intuition is transitive: a shortlisted point near the query is itself near the true neighbor, so the true neighbor tends to appear among that point’s stored nearest neighbors even when its own cell was not probed. This adds coverage that is *independent of the partition* — the property that drives the coarse-cell corollary (§8). It composes with γ : the two attack different misses (calibration fixes which cells are ranked highly; the graph reaches beyond the probed cells), overlapping only partially. The union is deduplicated against the scanned pool (already-scanned neighbors are not re-scored) and the adjacency reads are software-prefetched, so the added cost is a small, bounded re-rank increment.

Effect. At 1M, composed with γ , the ScaNN ratio drops $1.18 \rightarrow 0.98$ (the first sub-1 result); at 10M it lets t_{surv} halve at matched recall. Graph parameters scale with n : fan-in M and pool depth grow from $M \approx 25$ at 1M to $M=64$ at 100M as distractors multiply.

8 Routing Cost and the Coarse-Cell Corollary

8.1 A routing cost model

Consider an L -level tree over K_f leaves with cell counts $C_0 < C_1 < \dots < C_{L-1} < C_L = K_f$ and per-level beam widths $b_0, \dots, b_{L-2}$ (the number of cells expanded when descending from level i to level $i+1$). Routing a query costs, in centroid distance evaluations,

$$\text{route}(q) = C_0 + \sum_{i=1}^{L-1} b_{i-1} \frac{C_i}{C_{i-1}}, \quad (2)$$

since the coarse level scores all C_0 centroids and level i expands the b_{i-1} best parents from level $i-1$, each contributing C_i/C_{i-1} children. The scan then reads $\text{scan}(q) = p \cdot (n/K_f) \cdot a_0$ candidates (p probed leaves $\times$ points/leaf $\times$ multi-assignment). For a 2-level tree (2) is $C_0 + b_0 K_f / C_0$, minimized at $C_0 = \sqrt{b_0 K_f}$ to give $2\sqrt{b_0 K_f} = O(\sqrt{K_f})$; with L levels and balanced branching this falls to $O(L K_f^{1/L})$. The two costs pull oppositely in K_f : finer leaves shrink the scan (n/K_f) but grow the routing. The beams b_i are the “routing aggression” knob — wider beams retain the true cell with higher probability at linear cost — and OOD queries, which route poorly, need wider beams (§14).

Table 2 gives geometry and routing cost by scale. Two levels suffice through 10M; at 100M a 2-level tree at its optimal $C_0 = \sqrt{b_0 K_f} \approx 16,400$ would cost $\approx 33,000$ evals, whereas the 3-level tree roughly halves this (14,336) — so the third level is introduced only at 100M, where routing is a large enough share of the query to matter.

8.2 Scaling laws for geometry

To set geometry at scale without per- n retuning, we sweep K_f at $n \in \{10^5, 3 \cdot 10^5, 10^6, 3 \cdot 10^6, 10^7\}$ (2-level, graph off), at each n finding the minimum p that reaches $\text{recall@10} \geq 0.90$ (deterministic, hence load-independent) and the resulting cost $\text{route} + \text{scan}$ from Eq. (2). The cost-minimizing geometry follows clean power laws (log-log fit):

$$K_f^* \sim n^{0.66}, \quad \frac{n}{K_f^*} (\text{pts/leaf}) \sim n^{0.34}, \quad p^* \sim n^{0.18}, \quad C_0^* \sim n^{0.33}.$$

The central law is $\text{pts/leaf} \sim n^{1/3}$: optimal leaves *grow* with n , because routing $O(\sqrt{K_f})$ rises while the scan $O(pn/K_f)$ is held down by the growing K_f — the coarse-cell corollary expressed as a scaling law. Probes

grow only as $n^{1/5}$. Extrapolating, $n=10^9$ calls for $K_f \approx 10^6$ (~ 1000 pts/leaf). These calibrate the 2-level baseline; graph coverage (§7) shifts the optimum coarser and a third routing level (cheaper, $O(K_f^{1/3})$) shifts it finer, so the 100M geometry is validated by direct A/B (§14) rather than read off the 2-level law.

8.3 The corollary

[TODO: Sweep cell granularity at 10M with the graph on: fine (Kf=262144), medium (65536), coarse (32768); medium/coarse WIN. Bracket the knee with four builds.] The cost model (2) explains *why*. Without graph coverage, recall demands fine K_f (small leaves make the true neighbor’s leaf likelier to be among the p probed), paying $O(\sqrt{K_f})$ routing. The k -NN-graph union (§7) supplies that coverage *independently* of K_f , so the tree can use *coarser* leaves — cheaper routing — at fixed recall. The optimum is a knee: too fine wastes routing, too coarse inflates the scan ($p \cdot n/K_f$). Empirically the knee sits near 150 points/leaf at 10M, and both finer and coarser builds lose to it.

Effect. At 10M, moving from the fine-cell to the 152-point-cell geometry takes the ScaNN ratio $1.01 \rightarrow 0.85$; routing evals/query drop $\sim 12,000 \rightarrow 6,144$.

9 Joint Cross-Level Partition Optimization (Tree-EM)

Hierarchical k -means trains each level greedily top-down, so a leaf centroid is optimized for its parent’s Voronoi cell, not for the query-time beam descent that actually reaches it. We instead optimize all levels jointly by EM against the search objective: the E-step assigns each sample point to the leaf its *beam descent* lands in (not its globally-nearest leaf), and the M-step recomputes every level’s centroids from those assignments. Iterating co-adapts the levels so the descent lands better. The E-step is itself an ANN query against the tree’s own centroids, so it runs at a shallow beam (a few probes, not the full query beam), making EM cheap; it is a build-time cost only and adds nothing at query time. It composes with γ and the graph (it improves the partition; they improve routing and coverage on top).

Effect. +0.6–1.0 recall@10 points at 10M at matched probe, QPS-neutral (build-time only).

10 Multi-Assignment (SOAR)

The remaining coverage lever is redundancy: assign each point to its top- a_0 cells rather than one, so a query reaching any of them finds the point. Naive replication puts near-duplicate copies in adjacent cells; SOAR instead chooses the extra cells to cover the *residual direction* of the first assignment, so the copies span different query directions and buy more coverage per unit storage. Realizing the gain required a correctness fix: the exact-re-rank shortlist must be deduplicated by original id *before* the survivor cap, otherwise the a_0 duplicate copies of a point crowd the fixed-size shortlist and starve distinct ids (recall then falls, and paradoxically decreases with more probes — the signature of the bug). With the fix, multi-assignment raises the routing-coverage ceiling.

Effect. $a_0=2$ reaches OOD coverage parity with the baseline partition; higher a_0 extends the high-recall frontier at a storage/scan cost (quantified in table).

11 Systems Techniques (Streaming)

[TODO: msturing-30M-clustered final_runbook; recall@10 under 1h + 8GB. Three levers: (i) batch-parallel insert (assignment parallelized over the batch, order-preserving, bit-identical); (ii) low-live adaptive p (grow probes so expected candidates $\geq$ threshold when the live set is small); (iii) a parallelization fix (the batched re-rank driver ran chunks serially $\rightarrow$ RAYON never engaged). Plus a compliance re-architecture: first-insert-batch cold start + periodic live-set retraining.]

Effect. Batch insert: 30M inserts 1818s $\rightarrow$ 406s ($4.5\times$), recall-identical. Adaptive p : worst step $0.737 \rightarrow 0.944$. Parallelization fix: $7.6\times$ at 8 threads. Compliant recall@10 = [pending: final NQ=10000], peak <8GB.

12 Negative Results (Evidence for the Localization)

[TODO: Report each with its measured null/loss:]

- **Query-aware routing** (train cells on the query distribution): *worse* than base-trained (0.76 vs 0.89 @ matched p) — undertrained + the cells where queries are dense don’t distribute base points well.
- **ADC routing** (4-bit-scan the centroids): recall-neutral but *no speed win* — the exact VNNI router is already at the compute floor.
- **Anisotropic/OPQ-rotated coarse codes for rank preservation**: do *not* make coarse codes rank-preserving; reorder depth is set by code bit-rate, and rotation/ η move it < 0.01 . Coverage-via-coarsening collapses (a scann-coverage-matched dpb=50 config yields recall 0.09).
- **RaBitQ**: no ranking edge over PQ at matched bytes.
- **Wider SIMD** (AVX-512 vpermw / 64-wide): scan is memory-bound, not shuffle-bound; $\leq +7\%$, sometimes negative (downclock/cross-lane).
- **Finer cells at matched probe**: lose recall (half the probe fraction).
- **PCA/LeanVec dim reduction on OOD**: near-isotropic embeddings; dead.

These cluster on the scan/code side and all fail; the §5–9 levers cluster on routing/coverage and all succeed — the localization made concrete.

13 Evaluation Methodology

We report *same-hardware ratios*, never cross-machine QPS. All comparisons run on one physical machine; each competitor is measured in the same session, seconds apart, at its own best operating point.

Protocol. For a target recall ($\text{recall@10} \geq 0.90$), we (i) fix each system’s operating point by an independent sweep, requiring both to clear the recall gate against the *same* exact float ground truth; (ii) alternate the two systems round to round (order flips) so neither owns a load window; (iii) take best-of- k within a round and report the *median of per-round ratios*; (iv) recompute recall from raw ids, never trusting a cached number. On a shared machine, load drift between two builds minutes apart exceeds the effects being measured, so only interleaved same-window ratios and load-independent recall are trusted.

Warm-resident semantics; pristine confirmation. Both systems measured warm and resident (leader-board serving); an early cold-spawn asymmetry is corrected by scoring each engine’s warm block. Headline numbers pause all other first-party jobs (SIGSTOP) and confirm across ≥ 20 rounds.

The baseline at its best. At 10M, where an *official* big-ANN recipe exists, ScaNN uses it verbatim (40k leaves, anisotropic AH/LUT16, residual quant, bfloat16 re-order, top-level partitioner) with its own leaves-to-search / re-order sweep including corner configs. At scales with no official recipe (1M, 100M) we instead give ScaNN its *speed-optimal* configuration, found by sweeping the leaf count: e.g. at 1M, ScaNN peaks at 1200 leaves (6.2k QPS@0.90), with both finer and coarser partitions slower — coarser because routing over more leaves outweighs the finer scan, mirroring our own coarse-cell result (§8). This matters: a convention-chosen leaf count can silently under- or over-partition the baseline. A win depending on a handicapped baseline is quarantined; we describe one (an under-leaved cached 10M index) and its correction.

Table 3: OOD text2image, recall@10-matched QPS ratio (ScaNN/ours; < 1 = our engine faster). Same-hardware, tight-pairwise, opponent at official config. [\[pending: 100M row\]](#).

Scale	1-thread	8-thread	recall@10 gate
1M	0.755	0.756	0.905
10M	0.827	0.833	0.901
100M	[pending:]	[pending:]	[pending:]

Table 4: Ablation ladder at 10M (ScaNN/ours ratio; each row adds one lever).

Configuration	ratio
γ only	1.24
+ graph coverage	1.01
+ tree-EM	0.92
+ coarse-cell geometry (knee)	0.85
+ pristine confirmation (20 rounds)	0.83

Build-time eligibility. The benchmark caps index build at **12 hours** on the 8-vCPU evaluation machine, so a baseline (or our own index) that cannot build in time is not merely slow — it is ineligible. This is a real constraint at 100M: a $\sqrt{n}$ -scaled 126k-leaf ScaNN would take ≈ 32 h at 8 vCPU (its partitioner k -means over the training sample), so the eligible ScaNN baseline is a coarser config that builds in time. Our indices build comfortably inside the limit and *faster than ScaNN at scale*: on 16 cores, 1M in 131s, 10M in 27min, 100M in 86min (≈ 3 h at 8 vCPU), versus ScaNN’s ≈ 1.5 h (10M) and ≈ 2 h (100M, eligible config). We therefore report build time alongside QPS (Table 3); the hierarchical- k -means+PQ build is cheaper than ScaNN’s partitioner- k -means+AH training, and the gap widens with n .

What we do not claim. Same-hardware ratios on one (shared) machine, not a standardized-VM submission; ScaNN is the *measured* opponent — gaps to binary-only leaderboard systems are *inference* from the public leaderboard.

14 Results

[TODO: Add per-lever isolation deltas; 8-thread scaling; streaming table (recall, peak mem, wall vs 1h budget). Note the ablation ordering is the actual development arc.]

15 Related Work

[TODO: ScaNN (anisotropic VQ), DiskANN/HNSW (graphs), FAISS IVF-PQ, big-ANN challenge, RaBitQ. Position OOD-specific routing as the gap; distinguish γ from residual quantization (orthogonal) and from query-aware training (shown to fail).]

16 Limitations

[TODO: Single (shared) machine; γ fit per dataset; the coarse-cell corollary is demonstrated on IP/OOD data and may not transfer to in-distribution L_2 ; the gap to binary-only leaderboard systems is inference; filter/sparse tracks untouched; standardized-VM submission is future work and the gating experiment for a leaderboard claim.]